```
{
"name": "WF2_Booking_Orchestrator_Error_Handler",
"nodes": [
{
"parameters": {},
"name": "Error Trigger",
"type": "n8n-nodes-base.errorTrigger",
"typeVersion": 1,
"position": [0, 0],
"id": "err_trigger_001"
},
{
"parameters": {
"jsCode": "// Extract error details and booking context for rollback\nconst errorData =
$input.first().json;\n\n// Get workflow context\nconst workflowId = errorData.workflow?.id ||
'UNKNOWN';\nconst workflowName = errorData.workflow?.name || 'UNKNOWN';\nconst executionId =
errorData.execution?.id || 'MANUAL';\nconst lastNode = errorData.execution?.lastNodeExecuted
|| '';\nconst errorMessage = errorData.execution?.error?.message || 'Unknown error';\n\n//
Try to extract booking context from input data\nconst inputData =
errorData.execution?.data?.result_data || {};\nconst bookingContext = {\n  provider_id:
inputData.provider_id || null,\n  service_id: inputData.service_id || null,\n  start_time:
inputData.start_time || null,\n  customer_id: inputData.customer_id || inputData.chat_id ||
null,\n  idempotency_key: inputData.idempotency_key || null\n};\n\n// Determine what needs to
be rolled back based on which node failed\nlet rollbackData = {\n  booking_id: null,\n
gcal_event_id: null,\n  lock_id: null,\n  lock_key: null,\n  reason:
`auto_rollback_from_${lastNode}`\n};\n\n// If failed after GCal creation, we need to rollback
GCal\nif (lastNode.includes('GCal') || lastNode.includes('Calendar')) {\n
rollbackData.gcal_event_id = inputData.gcal_event_id || inputData.eventId || null;\n}\n\n//
If failed after DB insert, we need to rollback DB\nif (lastNode.includes('DB') ||
lastNode.includes('Postgres')) {\n  rollbackData.booking_id = inputData.booking_id ||
null;\n}\n\n// If failed after lock acquisition, we need to release lock\nif
(lastNode.includes('Lock')) {\n  rollbackData.lock_id = inputData.lock_id || null;\n
rollbackData.lock_key = inputData.lock_key || null;\n}\n\nreturn [{\n  json: {\n
workflow_id: workflowId,\n    workflow_name: workflowName,\n    execution_id: executionId,\n
error_message: errorMessage,\n    last_node: lastNode,\n    booking_context:
bookingContext,\n    rollback_data: rollbackData,\n    timestamp: new Date().toISOString()\n
}\n}];"
},
"name": "Extract Error Context",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [250, 0],
"id": "err_extract_002"
},
{
"parameters": {
"jsCode": "// Build rollback payload for WF6\nconst input = $input.first().json;\nconst
rollback = input.rollback_data;\nconst context = input.booking_context;\n\n// Use context to
fill in missing rollback data\nif (!rollback.booking_id && context.idempotency_key) {\n  //
We don't have booking_id yet, but we can try to find it\n  rollback.booking_id = null; //
Will be looked up by WF6 if needed\n}\n\n// Build lock_key from context if not available\nif
(!rollback.lock_key && context.provider_id && context.start_time) {\n  rollback.lock_key =
`lock_${context.provider_id}_${context.start_time}`;\n}\n\nreturn [{\n  json: {\n
...rollback,\n    _meta: {\n      source: 'WF2_ERROR_HANDLER',\n      original_workflow:
input.workflow_name,\n      original_execution: input.execution_id,\n      failed_at_node:
input.last_node,\n      timestamp: new Date().toISOString()\n    }\n  }\n}];"
},
```

"name": "Build Rollback Payload",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [500, 0],
"id": "err_build_003"
},
{
"parameters": {
"url": "https://n8n.stax.ink/webhook/rollback-booking",
"method": "POST",
"sendBody": true,
"bodyParameters": {
"parameters": [
{
"name": "booking_id",
"value": "={{ $json.booking_id }}"
},
{
"name": "gcal_event_id",
"value": "={{ $json.gcal_event_id }}"
},
{
"name": "lock_id",
"value": "={{ $json.lock_id }}"
},
{
"name": "lock_key",
"value": "={{ $json.lock_key }}"
},
{
"name": "reason",
"value": "={{ $json.reason }}"
}
]
},
"options": {
"timeout": 30000,
"retryOnFail": true,
"maxRetries": 3,
"retryInterval": 2000
}
},
"name": "Trigger WF6 Rollback",
"type": "n8n-nodes-base.httpRequest",
"typeVersion": 4.4,
"position": [750, 0],
"id": "err_trigger_wf6_004"
},
{
"parameters": {
"jsCode": "// Log the error and rollback result\nconst input = $input.first().json;\nconst errorContext = $('Extract Error Context').first().json;\n\nreturn [{\n json: {\n   success: true,\n   error_code: null,\n   error_message: null,\n   data: {\n   error_logged: true,\n   rollback_triggered: true,\n   original_error: errorContext.error_message,\n   failed_node: errorContext.last_node,\n   rollback_result: input\n   },\n   _meta: {\n   source: 'WF2_ERROR_HANDLER',\n   timestamp: new Date().toISOString(),\n   version: '1.0.0'\n   }\n }\n}];"
},

```
"name": "Log Error and Rollback",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
```

```
"position": [1000, 0],
"id": "err_log_005"
},
{
"parameters": {
"operation": "executeQuery",
"query": "INSERT INTO system_logs (workflow_name, execution_id, level, message, details)
VALUES ($1::text, $2::text, 'ERROR', $3::text, $4::jsonb) RETURNING id;",
"options": {
"queryReplacement": "={{ $json.workflow_name || 'WF2_ERROR_HANDLER' }}, {{ $json.execution_id
|| 'AUTO' }}, ERROR, {{ $json.error_message || 'Auto-rollback triggered' }}, {{
JSON.stringify($json) }}"
}
},
"name": "Log to DB",
"type": "n8n-nodes-base.postgres",
"typeVersion": 2.6,
"position": [1250, 0],
"credentials": {
"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}
},
"continueOnFail": true,
"id": "err_db_log_006"
}
],
"connections": {
"Error Trigger": {
"main": [
[
{
"node": "Extract Error Context",
"type": "main",
"index": 0
}
]
]
},
"Extract Error Context": {
"main": [
[
{
"node": "Build Rollback Payload",
"type": "main",
"index": 0
}
]
]
},
"Build Rollback Payload": {
"main": [
[
```

```
{
"node": "Trigger WF6 Rollback",
"type": "main",
"index": 0
}
]
]
},
"Trigger WF6 Rollback": {
"main": [
[
{
"node": "Log Error and Rollback",
"type": "main",
"index": 0
}
]
]
},
"Log Error and Rollback": {
"main": [
[
{
"node": "Log to DB",
"type": "main",
"index": 0
}
]
]
```

```
}
},
"settings": {
"executionOrder": "v1"
},
"id": "wf2-error-handler-auto"
}
```